BÖLÜM 5 / MODULE 5

Thread Havuzları, Executor'lar & Work-Stealing

Thread Pools, Executors & Work-Stealing Algorithms

ThreadPoolExecutor iç yapısı, 4 ret politikası (Rejection Policies), Executors sınıfının üretim felaketleri (OOM riskleri), ScheduledThreadPool, Fork/Join work-stealing algoritması ve Graceful Shutdown.

ThreadPoolExecutor mechanics, the 4 task rejection policies, production OOM hazards of Executors factory methods, ScheduledExecutor delay drift, Fork/Join work-stealing, and graceful shutdown.

25

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

ThreadPoolExecutor Anatomisi: CorePool, MaxPool, Kuyruk & Büyüme Algoritması

ThreadPoolExecutor Anatomy: Core vs Max Pool, Queues & Growth Dynamics

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              THREADPOOLEXECUTOR GÖREV KABUL KARAR AĞACI  │
└──────────────────────────────────────────────────────────┘

YENİ BİR GÖREV GELDİ (execute(task)):

1. ADIM: Çalışan thread sayısı < corePoolSize mı?
   ├──► EVET: Hemen yeni bir Core Thread aç ve görevi başlat!
   └──► HAYIR: 2. Adıma geç.

2. ADIM: BlockingQueue (İş Kuyruğu) dolu mu?
   ├──► HAYIR: Görevi kuyruğa at (Mevcut boştaki işçiler alsın).
   └──► EVET: 3. Adıma geç.

3. ADIM: Çalışan thread sayısı < maximumPoolSize mı?
   ├──► EVET: Hemen geçici bir Max Thread aç ve görevi başlat!
   └──► HAYIR: 4. Adıma geç.

4. ADIM: REJECTION POLICY (RET POLİTİKASI) DEVREYE GİRER!
   └──► (Abort, CallerRuns, Discard veya DiscardOldest)

⚠ EN BÜYÜK YANILGI: Havuz doğrudan maximumPoolSize'a kadar büyümez!
Önce kuyruk DOLMAK zorundadır! Kuyruk dolmadan ek thread ASLA açılmaz!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kargo şubesini düşünün. Şubede 3 daimi kadrolu personel (corePoolSize) çalışır. İlk 3 kargo gelince hemen bu personellere verilir. 4. kargo geldiğinde müdür yeni personel tutmaz; kargoyu arkadaki depoya (BlockingQueue) koyar! Ancak depo ağzına kadar dolup taştığında (Kuyruk dolduğunda), müdür geçici acil durum personelleri (maximumPoolSize) tutmaya başlar. Eğer depo da dolu, tüm geçici personeller de meşgulse; kapıya gelen müşteriye 'Kargonuzu alamıyoruz' denir (Rejection Policy)!

🇬🇧 ENGLISH (ELI5)

Think of a busy post office. It employs 3 full-time clerks (`corePoolSize`). The first 3 parcels go directly to them. When the 4th parcel arrives, the manager does NOT hire new staff; the parcel is placed into the backroom storage shelf (`BlockingQueue`). Only when that shelf is 100% physically packed full does the manager hire auxiliary temporary workers (`maximumPoolSize`). If temporary workers hit the legal ceiling AND the shelf is overflowing, incoming parcels are turned away at the door (`RejectedExecutionHandler`)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ThreadPoolExecutor Parametreleri ve HotSpot Yaşam Döngüsü: 1) corePoolSize vs maximumPoolSize: Birçok mühendis görevler geldikçe havuzun `maximumPoolSize`'a kadar büyüyeceğini sanır; bu yanlıştır! Standart mantıkta görev önce kuyruğa konur. Yalnızca kuyruk DOLDUĞUNDA `maximumPoolSize`'a kadar yeni thread açılır. 2) keepAliveTime ve Unit: Boşta kalan (idle) geçici thread'lerin ne kadar süre sonra sonlandırılacağını belirler (`allowCoreThreadTimeOut(true)` ile core thread'ler de sonlandırılabilir). 3) ctl Atomik Değişkeni: Doug Lea, havuz durumunu (RUNNING, SHUTDOWN, STOP, TIDYING, TERMINATED) ve aktif thread sayısını tek bir 32-bit `AtomicInteger ctl` içinde bit-masking ile tutar (Üst 3 bit durum, alt 29 bit thread sayısı).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ThreadPoolExecutor Architecture & Bit-Packing: 1) The Growth Order Axiom: Tasks route to: `corePool -> workQueue -> maxPool -> RejectedExecutionHandler`. Extra threads past `corePoolSize` are ONLY created if the queue rejects the `offer()`. 2) Atomic Control Word (`ctl`): HotSpot uses a single `AtomicInteger ctl` to pack both worker count and run state. The high 3 bits represent states (RUNNING = −1, SHUTDOWN = 0, STOP = 1, TIDYING = 2, TERMINATED = 3); the lower 29 bits encode the active worker count (supporting up to 536 million threads). 3) Worker Encapsulation: Internal `Worker` classes extend AQS to implement non-reentrant mutual exclusion locks, preventing interrupts during active task execution.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Üretim Kalitesinde ThreadPoolExecutor Konfigürasyonu
import java.util.concurrent.*;

public class ProductionThreadPoolFactory {
    public static ThreadPoolExecutor createCustomPool() {
        int corePoolSize = 4;
        int maxPoolSize = 8;
        long keepAliveTime = 60L;

        // ⚠ Sınırlı (Bounded) kuyruk OOM'u kesinlikle önler!
        BlockingQueue<Runnable> queue = new ArrayBlockingQueue<>(100);

        ThreadFactory threadFactory = Thread.ofPlatform()
            .name("order-processor-", 1)
            .factory();

        // 4. Adım: Kuyruk ve Max dolunca çağıran thread çalıştırsın (CallerRuns)
        RejectedExecutionHandler handler = new ThreadPoolExecutor.CallerRunsPolicy();

        return new ThreadPoolExecutor(
            corePoolSize,
            maxPoolSize,
            keepAliveTime,
            TimeUnit.SECONDS,
            queue,
            threadFactory,
            handler
        );
    }

    void main() {
        var pool = createCustomPool();
        pool.execute(() -> IO.println("Görev çalıştı: " + Thread.currentThread().getName()));
        pool.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı Sorusudur: 'corePoolSize=2, maxPoolSize=10, queueCapacity=100 olan bir havuza aynı anda 50 görev gönderirsek kaç thread açılır?' Yanıt: 'SADECE 2 THREAD AÇILIR! Çünkü ilk 2 görev core thread'leri alır, kalan 48 görev 100 kapasiteli kuyruğa atılır. Kuyruk dolmadığı için 3. thread bile açılmaz!' Bu soruyu bilen aday doğrudan öne geçer.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

The 1 MAANG Pool Question: 'If core=2, max=10, and queueCapacity=100, and 50 tasks arrive concurrently, how many threads exist?' Answer: 'EXACTLY 2 THREADS! The first 2 tasks spawn the core workers. The remaining 48 tasks are buffered in the 100-capacity queue. Because the queue never overflows, temporary workers are never spawned.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Havuz kuyruk dolmadan maximumPoolSize'a kadar ASLA genişlemez.
- ctl değişkeni tek bir 32-bit integer içinde hem havuz durumunu hem thread sayısını tutar.
- Üretimde sınırsız (unbounded) kuyruk kullanmak OOM çöküşlerine davetiyedir.

🎯 Key Takeaways for the Interview

- Pools never expand past corePoolSize until the bounded work queue is 100% full.
- HotSpot packs pool run state and thread count into a single 32-bit atomic integer.
- Unbounded work queues cause OutOfMemoryErrors under sustained traffic spikes.

26

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Ret Politikaları (Rejection Policies): Abort, CallerRuns, Discard & DiscardOldest

Task Rejection Policies: AbortPolicy, CallerRuns, Discard & DiscardOldest

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              THREAD POOL 4 RET POLİTİKASI (REJECTION POLICIES)              │
└──────────────────────────────────────────────────────────┘

KUYRUK DOLU VE MAX THREAD'LER MEŞGUL İKEN YENİ GÖREV GELDİĞİNDE:

1. AbortPolicy (Varsayılan / Sert):
   ──► RejectedExecutionException fırlatır! "Ben tıkandım, işlemi reddediyorum!"

2. CallerRunsPolicy (En Popüler / Doğal Backpressure):
   ──► Görevi havuz değil, görevi havuza GÖNDEREN THREAD (Örn: Tomcat HTTP) çalıştırır!
   ──► Gönderen thread meşgul olunca yeni istek kabul edemez; sistem doğal olarak yavaşlar!

3. DiscardPolicy (Sessiz Çöp):
   ──► Görevi hiçbir hata fırlatmadan SESSİZCE ÇÖPE ATAR! (Örn: Önemsiz log/metrik)

4. DiscardOldestPolicy (En Eskiye Tekme):
   ──► Kuyruğun başındaki en eski görevi silip atar, yeni görevi kuyruğa sokar!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir pizzacı dükkanı düşünün. Fırınlar dolu, masalar dolu, kuryeler meşgul. Kapıdan yeni bir müşteri girip pizza istediğinde pizzacı ne yapar? 1) Abort: Müşterinin yüzüne kapıyı çarpar: 'Sipariş alamam!' der (Hata fırlatır). 2) CallerRuns: Müşterinin eline önlük tutuşturur: 'Çok istiyorsan mutfağa gir kendin pişir!' der (İsteği getiren kişi kendi işini kendi yapar, böylece kapıda yeni müşteri birikmez). 3) Discard: Sipariş fişini alır ve müşteri arkasını döndüğü anda çöpe atar (Sessizce yutar). 4) DiscardOldest: Bekleyen en eski müşterinin siparişini iptal eder, yeni gelen müşteriyi sıraya alır!

🇬🇧 ENGLISH (ELI5)

Imagine a bakery operating at maximum capacity. The oven is full, counters overflowing. When a customer orders a cake: 1) AbortPolicy: The baker slams the shutter and shouts 'Denied!' (`RejectedExecutionException`). 2) CallerRunsPolicy: The baker hands the customer an apron: 'Bake it yourself!' The caller executes the task on their own thread, naturally throttling new orders. 3) DiscardPolicy: The baker silently drops the order slip into the trash (silent drop). 4) DiscardOldestPolicy: The baker evicts the oldest stale ticket waiting in line and enqueues the fresh order.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CallerRunsPolicy ile Doğal Backpressure (Geri Basınç): 1) Neden Üretimde CallerRuns Tercih Edilir?: Web sunucularında (Tomcat, Netty, Spring Boot) arka plandaki thread havuzu tıkandığında `CallerRunsPolicy` seçilirse, görevi havuza atan HTTP Acceptor thread'i görevi kendi üzerinde koşturmaya başlar. Bu süre zarfında soketten yeni HTTP istekleri kabul edilemez; işletim sistemi TCP SYN kuyruğunda bekletir veya yük dengeleyiciye (Load Balancer) 'Ben doluyum' sinyali gider. Bu, sistemin OOM çöküşünü önleyen en zarif mimari savunmadır. 2) DiscardOldest Tehlikesi: `PriorityBlockingQueue` ile kullanılırsa, en eski görev yerine en yüksek öncelikli görev çöpe atılabilir!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CallerRunsPolicy & The Elegance of Organic Backpressure: 1) Mitigating Cascading Failures: When a downstream worker pool saturates, `CallerRunsPolicy` delegates execution to the invoking thread (e.g. Tomcat/Jetty worker). Because the calling thread is now occupied executing computation, it stops accepting new requests from the network socket. The TCP receive buffer fills, and upstream load balancers detect backpressure, rerouting traffic away from the saturated node. 2) DiscardOldest Hazard: When paired with a `PriorityBlockingQueue`, `DiscardOldest` evicts the head of the queue, which is the HIGHEST priority task, not the oldest chronologically!

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: CallerRunsPolicy ile Doğal Backpressure Gösterimi
import java.util.concurrent.*;

public class RejectionPolicyDemo {
    void main() {
        // 1 Thread, 1 Kuyruk Kapasitesi (Toplam 2 görev alabilir)
        var executor = new ThreadPoolExecutor(
            1, 1, 0L, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(1),
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure kalkanı
        );

        for (int i = 1; i <= 4; i++) {
            final int taskId = i;
            executor.execute(() -> {
                IO.println("Görev #" + taskId + " çalıştırılıyor. Thread: " +
Thread.currentThread().getName());
                try { Thread.sleep(200); } catch (InterruptedException ignored) {}
            });
        }

        executor.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'CallerRunsPolicy kullandığımızda ana thread kilitlenebilir mi?' diye sorar. Yanıt: 'Evet! Eğer havuza atılan görev uzun süren bir işlemse veya bloke oluyorsa, ana thread (ör. UI thread veya Netty event-loop) o görevi çalıştırırken donar ve tüm uygulamanın tepki süresi (responsiveness) durur!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Architectural Hazard: 'Can CallerRunsPolicy degrade the application main loop?' Answer: 'YES! If the calling thread is a Netty EventLoop or a UI dispatcher, forcing it to run long-running CPU or blocking I/O tasks halts the entire event-loop, starving all other multiplexed channels.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• AbortPolicy varsayılandır ve RejectedExecutionException fırlatır.

• CallerRunsPolicy, çağıran thread'i çalıştırarak doğal bir backpressure kalkanı oluşturur.

• DiscardPolicy sessizce görevi yutar; yalnızca önemsiz telemetri loglarında kullanılmalıdır.

🎯 Key Takeaways for the Interview

• AbortPolicy is the default, throwing RejectedExecutionException on saturation.

• CallerRunsPolicy creates natural backpressure by forcing the submitting thread to execute the work.

• DiscardPolicy silently drops tasks and should only be applied to dispensable metrics.

27

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Executors Fabrika Tuzakları: newFixedThreadPool Neden Üretimde OOM Yaratır?

Executors Factory Traps: Why newFixedThreadPool Triggers Production OOM

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              EXECUTORS.NEWFIXEDTHREADPOOL() ÖLÜMCÜL KODU     │
└──────────────────────────────────────────────────────────────┘

Executors.newFixedThreadPool(10) ÇAĞRISININ ARKA PLANI:

return new ThreadPoolExecutor(
    10, 10, 0L, TimeUnit.MILLISECONDS,
    new LinkedBlockingQueue<Runnable>() // 💣 SAATLİ BOMBA BURADA!
);

LinkedBlockingQueue'nun Varsayılan Kapasitesi:
Integer.MAX_VALUE = 2,147,483,647 (2 MİLYAR GÖREV!)

GÖREVLER HAVUZA AKARKEN:
10 Core Thread kilitlenir veya yavaşlar ──► Gelen görevler kuyrukta birikir:
[Task 1, Task 2, Task 3, ... Task 1,000,000, ... Task 5,000,000]
                                        │
                                        ▼
HEAP BELLEK BİTER: java.lang.OutOfMemoryError: Java heap space ──► SİSTEM ÇÖKER!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir mağaza açıyorsunuz. İçeriye 10 tezgahar koyuyorsunuz (newFixedThreadPool(10)). Ama bekleme salonunu 'sonsuz' büyüklükte yapıyorsunuz (2 milyar kişilik kuyruk)! Tezgahlarda işler biraz yavaşladığında kapıdaki 5 milyon insan mağazanın koridorlarına doluşur. Sonunda binanın kolonları bu ağırlığı taşıyamaz ve bina herkesin üstüne çöker (OutOfMemoryError)! İşte `Executors.newFixedThreadPool` kullanmak, mağazanın kapısına sınırsız insan alıp binayı çökertmektir!

🇬🇧 ENGLISH (ELI5)

Imagine opening a bank branch with 10 tellers (`newFixedThreadPool(10)`). However, you build a waiting lobby capable of holding 2.14 billion humans (`LinkedBlockingQueue`). If the tellers suffer an external network delay, millions of customers flood into the building. Eventually, the floor collapses under the physical weight, demolishing the entire bank (`OutOfMemoryError`). Never configure an infinite lobby!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Executors Metotlarının MAANG Üretim Analizi: 1) newFixedThreadPool & newSingleThreadExecutor Felaketi: İkisi de unbounded `LinkedBlockingQueue` (kapasite: 2^31-1) kullanır. Tüketici hızı üretici hızından yavaş olduğunda görev nesneleri Heap'i doldurur ve JVM OOM ile çöker. 2) newCachedThreadPool Felaketi: `corePoolSize = 0`, `maximumPoolSize = Integer.MAX_VALUE` ve `SynchronousQueue` kullanır. Yoğun trafikte her gelen istek için yeni bir OS thread'i açmaya çalışır; binlerce thread açıldığında işletim sistemi seviyesinde `java.lang.OutOfMemoryError: unable to create new native thread` patlar. 3) MAANG Kuralı: `Executors` statik fabrikalarının kullanımı kurumsal kod tabanlarında kesinlikle yasaklanmalıdır (Checkstyle / SonarQube kuralı); daima parametreleri açıkça belirlenmiş `ThreadPoolExecutor` kurulmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Why Static Executors Factories are Banned at MAANG: 1) `newFixedThreadPool` Unbounded Queue Doom: It wraps an unbounded `LinkedBlockingQueue` (capacity `Integer.MAX_VALUE`). Under downstream latency spikes, tasks accumulate on heap memory until a catastrophic `OutOfMemoryError: Java heap space` kills the JVM process. 2) `newCachedThreadPool` Thread Explosion Doom: It sets `maximumPoolSize = Integer.MAX_VALUE` over a zero-capacity `SynchronousQueue`. In traffic surges, it spawns thousands of OS platform threads until memory or OS limits trip, crashing with `OutOfMemoryError: unable to create native thread`. 3) Staff+ Guideline: Enforce linter rules prohibiting `java.util.concurrent.Executors` factory methods. Always instantiate `ThreadPoolExecutor` with explicit bounded queues and custom rejection handlers.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Bounded Havuz vs Tehlikeli Executors Karşılaştırması
import java.util.concurrent.*;

public class SafeVsUnsafePools {
    // ❌ TEHLİKELİ (Üretimde KESİNLİKLE kullanmayın!):
    public static ExecutorService createDangerousPool() {
        return Executors.newFixedThreadPool(10); // Unbounded kuyruk = OOM riski!
    }

    // ✔ GÜVENLİ (MAANG Standardı):
    public static ExecutorService createEnterpriseSafePool() {
        return new ThreadPoolExecutor(
            8,                          // Core pool size
            16,                         // Max pool size
            30L, TimeUnit.SECONDS,      // Idle timeout
            new ArrayBlockingQueue<>(5000), // ⚠ Sınırlı Kuyruk (Bounded Queue)
            Thread.ofPlatform().name("safe-worker-", 1).factory(),
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure
        );
    }

    void main() {
        var safePool = createEnterpriseSafePool();
        safePool.execute(() -> IO.println("Güvenli kurumsal havuz aktif!"));
        safePool.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Executors.newCachedThreadPool() ne zaman çöker?' Yanıt: 'Gelen istek hızı thread'lerin işi bitirme hızından yüksek olduğunda, havuz sonsuza kadar yeni OS thread'i açmaya çalışır. 10.000 thread açıldığında sistem RAM'i tükenir ve unable to create new native thread hatasıyla çöker!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Distinction: 'What is the failure mode of `newCachedThreadPool`?' Senior Answer: 'Unbounded thread creation! Because maximumPoolSize is Integer.MAX_VALUE, any traffic burst spawns thousands of native threads. Each consumes 1MB stack plus OS kernel metadata, crashing the JVM with `OutOfMemoryError: unable to create new native thread`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- newFixedThreadPool sınırsız LinkedBlockingQueue yüzünden Heap OOM yaratır.
- newCachedThreadPool sınırsız thread açtığı için Native Thread OOM patlatır.
- Üretimde DAİMA sınırlı kuyruklu (ArrayBlockingQueue) açık ThreadPoolExecutor kullanılır.

🎯 Key Takeaways for the Interview

- newFixedThreadPool uses an unbounded LinkedBlockingQueue, causing Heap OOM crashes.
- newCachedThreadPool spawns infinite threads, causing Native Thread OOM panics.
- In production, always construct explicit ThreadPoolExecutors with bounded queues.

28

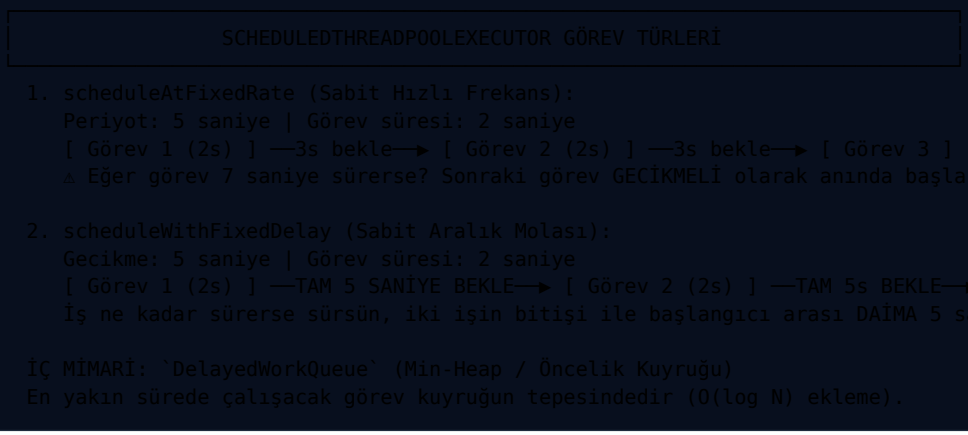
BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

ScheduledThreadPoolExecutor: Zamanlanmış Görevler, Drift & İptal Mekanizmaları

ScheduledThreadPoolExecutor: Timer Scheduling, Delay Drift & Task Cancellation

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki tip ilaç saati gibi düşünün: 1) FixedRate (Sabit Saat): Doktor 'Her sabah saat tam 08:00'de ilaç iç' dedi. Kahvaltıyı uzatsanız bile saat 08:00 olunca ilacı almak zorundasınız. 2) FixedDelay (Sabit Mola): Doktor 'Yemeğin bittikten TAM 2 saat sonra ilacını iç' dedi. Kahvaltı ister 10 dakika ister 1 saat sürsün; kahvaltı bittiği andan itibaren kronometre 2 saat saymaya başlar! Eğer bir işin önceki iş bittikten sonra belirli bir süre dinlenmesini istiyorsanız `scheduleWithFixedDelay` seçmelisiniz!

🇬🇧 ENGLISH (ELI5)

Compare two medicine schedules: 1) `scheduleAtFixedRate`: 'Take medicine at 08:00, 12:00, 16:00'. If lunch runs late, the schedule still demands delivery at the clock strike. 2) `scheduleWithFixedDelay`: 'Take medicine exactly 4 hours AFTER finishing your previous meal'. The timer starts only when the prior task completes. For periodic database synchronization or cleanup jobs, `scheduleWithFixedDelay` prevents overlapping runs if a task takes longer than expected!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ScheduledThreadPoolExecutor İç Mimarisi ve İstisna Tuzağı: 1) DelayedWorkQueue (Min-Heap): Görevler `ScheduledFutureTask` nesnelerine sarılır. Kuyruk, çalışma zamanına göre sıralanmış bir ikili yığın (Binary Min-Heap) tutar. 2) Sessiz Ölüm Tuzağı (Silent Death): Periyodik çalışan bir görev (`Runnable`) içinde yakalanmamış bir `RuntimeException` fırlarsa, `ScheduledExecutorService` o görevi SESSİZCE İPTAL EDER ve bir daha asla çalıştırmaz! Konsola hiçbir hata basılmaz! Görev arka planda ölmüştür ve kimsenin haberi olmaz. 3) Çözüm: Periyodik görevlerin `run()` metodu en dışta `try-catch(Throwable t)` ile sarılmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ScheduledExecutor Deep-Dive & The Silent Death Hazard: 1) DelayedWorkQueue Topology: Backed by a concurrent Fibonacci/Min-Heap, ordering scheduled tasks by trigger epoch (`getDelay(TimeUnit.NANOSECONDS)`). 2) FixedRate vs FixedDelay Under Overrun: If a task under `scheduleAtFixedRate` exceeds the interval, subsequent executions do not overlap (single-thread constraint per task instance), but they stack up without pause, causing high CPU storms. `scheduleWithFixedDelay` strictly guarantees `delay` elapsed time between execution completion and next launch. 3) ⚠️ The Fatal Silent Death Trap: If an unhandled `Throwable` escapes a scheduled Runnable, the executor halts all future executions of that task permanently, suppressing the stack trace! Always envelop scheduled loops with `try-catch (Throwable t)`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Periyodik Görev Zamanlayıcı
import java.util.concurrent.*;

public class ResilientScheduler {
    void main() {
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

        // Güvenli Periyodik Görev (FixedDelay)
        scheduler.scheduleWithFixedDelay(() -> {
            try {
                IO.println("Cache temizliği başladı: " + System.currentTimeMillis());
                Thread.sleep(100);
            } catch (Throwable t) {
                // ⚠ KESİNLİKLE THROWABLE YAKALANMALIDIR! Aksi halde scheduler görevi öldürür!
                IO.println("Hata yakalandı, görev ölmedi: " + t.getMessage());
            }
        }, 0, 1, TimeUnit.SECONDS);

        // 3 saniye sonra durdur
        scheduler.schedule(() -> {
            IO.println("Scheduler kapatılıyor...");
            scheduler.shutdown();
        }, 3, TimeUnit.SECONDS);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Üretim Kabusu: 'ScheduledExecutorService'e attığım cron görevi 3 gün çalıştıktan sonra durdu, hata da vermedi, neden?' Yanıt: 'Görev içinde bir NullPointerException fırlamıştır ve catch edilmemiştir. Scheduler görevi kalıcı olarak iptal etmiştir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Real-World Production Horror: 'Our periodic cache refresher ran for 4 days then mysteriously vanished without crashing the JVM. Why?' Answer: 'An uncaught NullPointerException escaped the Runnable. ScheduledExecutorService silently terminated future iterations without logging to stdout.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- scheduleAtFixedRate sabit frekansla çalışır; scheduleWithFixedDelay iş bittikten sonra bekler.
- Yakalanamayan RuntimeException periyodik görevleri SESSİZCE ÖLDÜRÜR.
- Periyodik görevlerin gövdesi daima try-catch(Throwable) ile sarılmalıdır.

🎯 Key Takeaways for the Interview

- scheduleAtFixedRate targets fixed intervals; scheduleWithFixedDelay enforces delay between runs.
- Uncaught exceptions silently cancel future executions of scheduled tasks permanently.
- Always wrap scheduled task bodies inside a defensive try-catch(Throwable) block.

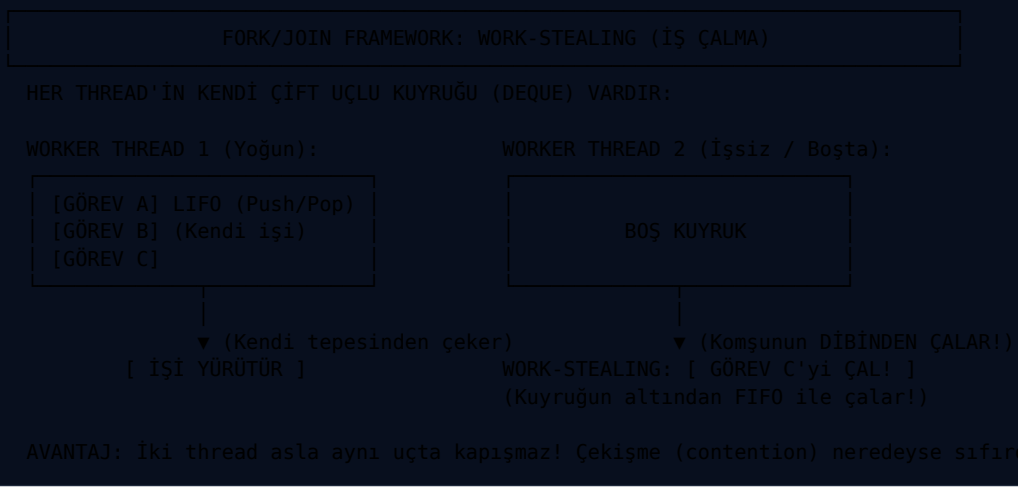
29

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Fork/Join Çerçevesi & Work-Stealing Algoritması: İş Çalma & Çift Uçlu Kuyruklar

Fork/Join Framework & The Work-Stealing Algorithm: Deque Poaching & Recursion

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki işçi düşünün: Ali ve Veli. İkisinin de önünde kendi özel iş tepsisi var. Ali'nin tepsisinde 100 tane evrak yığılı, Veli ise kendi işlerini hemen bitirip boşta kalmış. Veli oturup tembellik yapmaz; hemen Ali'nin tepsisine uzanır! Ama Ali tepsisinin tepesinden en taze evrakı alırken, Veli tepsisinin en altındaki en eski evrakı çalar (Work-Stealing). İkisi tepsisinin farklı uçlarından tuttukları için birbirlerinin eline çarpmazlar! Böylece hiçbir CPU çekirdeği boş kalmaz!

🇬🇧 ENGLISH (ELI5)

Imagine two postal sorters: Alice and Bob. Each owns a private Double-Ended Queue (Deque). Alice has 100 letters stacked; Bob clears his pile and sits idle. Under the Work-Stealing algorithm, Bob does not stay idle. He peeks at Alice's tray! Alice pushes and pops small subtasks from the TOP of her deque (LIFO, cache-friendly). Bob steals large root tasks from the BOTTOM of her deque (FIFO). Because they operate on opposite ends, thread contention is mathematically minimized!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ForkJoinPool ve Work-Stealing Derinliği: 1) Doug Lea'nın Work-Stealing Algoritması: Her worker thread kendi `WorkQueue` (çift uçlu kuyruk - Deque) yapısına sahiptir. Kendi görevlerini LIFO (Last-In-First-Out) sırasıyla işler; bu sayede CPU L1 önbellek uyumluluğu (Cache Locality) maksimize edilir. 2) Hırsızlık (Stealing): Boşta kalan thread başka bir rastgele worker'ın kuyruk dibinden FIFO ile çalar; en büyük iş parçacığı genellikle dipte yattığı için çalınan iş hırsızı uzun süre meşgul eder. 3) Java Paralel Akışlar (Parallel Streams): `list.parallelStream()` arka planda ortak `ForkJoinPool.commonPool()`'u kullanır! Eğer paralel stream içinde bloklayıcı bir I/O (ör. HTTP isteği veya DB sorgusu) yaparsanız, ortak havuzdaki tüm çekirdekleri kilitler ve tüm JVM'i çökertirsiniz!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ForkJoinPool Mechanics & The Parallel Streams Hazard: 1) Dual-Ended Contention Isolation: Workers interact with their local array deque via LIFO push/pop, minimizing synchronization and maximizing CPU cache locality. Idle workers steal from victim deques via FIFO CAS operations on the opposite tail end. Large coarse-grained un-split tasks naturally reside at the tail, giving thieves substantial work chunks. 2) RecursiveTask vs RecursiveAction: Subclasses override `compute()`. `fork()` pushes asynchronously; `join()` executes cooperative work while waiting rather than blocking. 3) The `ForkJoinPool.commonPool()` Anti-Pattern: Java Parallel Streams execute on a shared global common pool sized to CPU cores. Executing blocking I/O inside parallel streams starves the common pool, deadlocking unrelated library components across the JVM.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Fork/Join ile Büyük Dizi Toplamı (Böl ve Yönet / Divide and Conquer)
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

public class ParallelSumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000; // Eşik değer
    private final long[] array;
    private final int start, end;

    public ParallelSumTask(long[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) sum += array[i];
            return sum;
        }

        int mid = start + (end - start) / 2;
        var leftTask = new ParallelSumTask(array, start, mid);
        var rightTask = new ParallelSumTask(array, mid, end);

        leftTask.fork(); // Sol parçayı başka bir thread'e çalması için bırak
        long rightResult = rightTask.compute(); // Sağ parçayı kendin hesapla
        long leftResult = leftTask.join();     // Sol parçayı bekle ve birleştir

        return leftResult + rightResult;
    }

    void main() {
        long[] data = new long[100_000];
        for (int i = 0; i < data.length; i++) data[i] = 1;

        try (var pool = new ForkJoinPool()) {
            long total = pool.invoke(new ParallelSumTask(data, 0, data.length));
            IO.println("Paralel Toplam Sonucu: " + total); // 100,000
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

MAANG Mülakat Kuralı: 'parallelStream() içine HTTP isteği veya veritabanı sorgusu yazabilir miyiz?' Yanıt: 'KESİNLİKLE HAYIR! parallelStream() tüm JVM'de paylaşılan ForkJoinPool.commonPool()'u kullanır. Birkaç yavaş I/O çağrısı havuzdaki tüm çekirdekleri tüketir ve sistemdeki diğer tüm paralel akışları felç eder!'

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Fatal Architecture Sin: 'Can we run HTTP calls or database queries inside parallelStream()?' Answer: 'NEVER! Parallel streams default to the shared ForkJoinPool.commonPool(). Blocking calls tie up common pool carrier threads, causing system-wide thread starvation across the entire JVM.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Fork/Join çerçevesi işlemci yoğun (CPU-bound) böl-ve-yönet algoritmaları içindir.

• Work-stealing, boşta kalan işçilerin diğer kuyrukların dibinden iş çalmasıdır.

• Bloklayıcı I/O işlemleri asla parallelStream() ve commonPool içine konulmamalıdır.

🎯 Key Takeaways for the Interview

• ForkJoinPool is optimized strictly for CPU-bound recursive divide-and-conquer workloads.

• Work-stealing allows idle workers to steal tasks from the tail of busy workers' deques.

• Never execute blocking I/O inside parallel streams or the shared common pool.

30

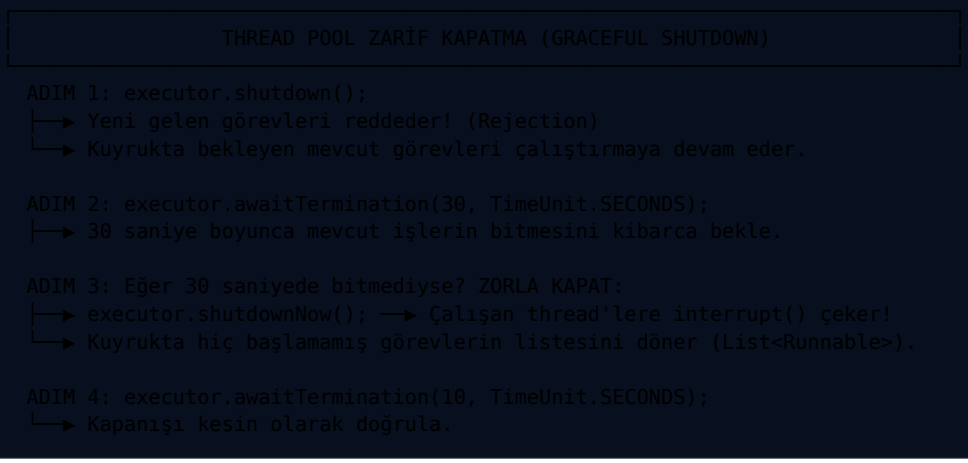
BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Zarif Kapatma (Graceful Shutdown): shutdown, shutdownNow & Zehirli Hap (Poison Pill)

Graceful Shutdown Patterns: shutdown(), shutdownNow() & The Poison Pill Pattern

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranın kapanış saati gibidir: 1. Adım (`shutdown()`): Kapı kilitlenir, yeni müşteri alınmaz. Ama masada yemeğini yiyen müşteriler kovulmaz; yemeklerini bitirmeleri beklenir. 2. Adım (`awaitTermination()`): Garsonlar masaların kalkmasını 30 dakika bekler. 3. Adım (`shutdownNow()`): 30 dakika geçmesine rağmen kalkmayan inatçı müşteri olursa ışıklar söndürülür, müzik kapatılır ve güvenlik herkesi dışarı çıkarır (Thread Interrupt)! İşte zarif kapatma, veri kaybı yaşamadan sistemi güvenle kapatma sanatıdır!

🇬🇧 ENGLISH (ELI5)

Shutting down a thread pool is like closing a restaurant: Step 1 (`shutdown()`): Lock the front door to new customers. Do not evict seated patrons eating dinner; let them finish. Step 2 (`awaitTermination()`): Wait patiently for 30 minutes for tables to clear. Step 3 (`shutdownNow()`): If stubborn diners refuse to leave after 30 minutes, cut the power, turn off the music, and usher them out via security (`Thread.interrupt()`). Graceful shutdown guarantees zero data loss.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üretim Seviyesinde 2-Aşamalı Kapatma Deseni (Two-Phase Termination): 1) `shutdown()` vs `shutdownNow()`: `shutdown()` havuzun durumunu SHUTDOWN yapar; yeni iş almaz ama kuyruktaki işleri tamamlar. `shutdownNow()` havuzu STOP durumuna çeker, kuyruktaki bekleyen görevleri çalıştırmadan bir `List\<Runnable>` olarak geri döner ve çalışan işçilere `interrupt()` sinyali gönderir. 2) Poison Pill Deseni: Kuyruk tabanlı Producer-Consumer sistemlerinde özel bir 'Zehirli Nesne' (ör. `POISON_PILL = new Task(-1)`) kuyruğa atılır. Tüketici bu nesneyi gördüğü anda döngüden çıkarak kendini sonlandırır; böylece `interrupt()` zorlamasına gerek kalmadan tamamen zarif bir kapanış sağlanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Oracle Standard 2-Phase Termination Protocol: 1) State Transitions: `shutdown()` transitions state from RUNNING to SHUTDOWN, refusing incoming tasks while executing buffered queue items. `shutdownNow()` transitions state directly to STOP, actively calling `Thread.interrupt()` on workers and draining unstarted tasks into a `List\<Runnable>` return payload. 2) The Poison Pill Architecture: In high-throughput messaging pipelines, a designated Sentinel Object (the Poison Pill) is injected into the queue. Consumers break their processing loops upon encountering the pill, achieving deterministic, exception-free termination without relying on thread interrupts.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Resilient Two-Phase Graceful Shutdown Deseni
import java.util.concurrent.*;

public class GracefulShutdownManager {
    public static void shutdownGracefully(ExecutorService pool, long timeoutSeconds) {
        pool.shutdown(); // 1. Yeni iş kabulünü durdur
        try {
            // 2. Mevcut işlerin bitmesini bekle
            if (!pool.awaitTermination(timeoutSeconds, TimeUnit.SECONDS)) {
                IO.println("⚠ İşler zamanında bitmedi, zorla kapatılıyor (shutdownNow)...");
                var droppedTasks = pool.shutdownNow(); // Çalışanlara interrupt çek
                IO.println("İptal edilen kuyruk görev sayısı: " + droppedTasks.size());

                if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
                    IO.println("❌ Havuz tamamen kapatılamadı!");
                }
            } else {
                IO.println("✔ Havuz tüm görevleri başarıyla tamamlayıp kapandı.");
            }
        } catch (InterruptedException e) {
            pool.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }

    void main() {
        var pool = Executors.newFixedThreadPool(2);
        pool.execute(() -> {
            try { Thread.sleep(500); } catch (InterruptedException ignored) {}
        });
        shutdownGracefully(pool, 2);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'shutdownNow() çağrıldığında çalışan thread anında pat diye durur mu?' Yanıt: 'HAYIR! Java'da thread'leri dışarıdan zorla öldüren bir mekanizma yoktur (Thread.stop() deprecated ve tehlikelidir). shutdownNow() yalnızca thread'e interrupt() sinyali gönderir; eğer çalışan kod InterruptedException yakalamıyor veya Thread.interrupted() kontrol etmiyorsa çalışmaya devam eder!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Java Truth: 'Does shutdownNow() kill running threads instantly?' Senior Answer: 'NO! Java has no safe pre-emptive thread-killing primitive (`Thread.stop()` is deprecated and dangerous). `shutdownNow()` merely issues a cooperative `interrupt()` signal. If the executing code does not poll `Thread.interrupted()` or sleep, it runs forever!'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- shutdown() kuyruktaki işleri bitirir; shutdownNow() çalışanlara interrupt çeker.
- Standart 2-aşamalı kapatma deseni: shutdown() -> awaitTermination() -> shutdownNow().
- Java thread'leri zorla öldürülemez; cooperative interruption mantığına uymak zorundadır.

🎯 Key Takeaways for the Interview

- shutdown() drains the queue; shutdownNow() interrupts active workers and returns unexecuted tasks.
- Always apply the 2-phase termination protocol: shutdown() -> await -> shutdownNow().
- Threads cannot be forcefully killed; they must cooperatively handle interruption.